File System 🗁

A **file system** is the method and data structure that an operating system uses to organize and manage files and directories on a storage device, such as a hard disk drive (HDD), solid-state drive (SSD), or USB drive.[1] It provides a logical, hierarchical view of the data, making it easy for users to access, store, and manipulate files without needing to know their physical location on the disk.

The file system acts as an interface between the user and the storage hardware.[2] Its main functions include:

- **File Management**: Creating, deleting, opening, reading, writing, and closing files.[3]
- **Directory Management**: Organizing files into a hierarchical directory structure (folders).[4]
- **Space Allocation**: Keeping track of which disk blocks are free and which are occupied by files.
- **Security and Protection**: Controlling user access and permissions to files and directories.[5]

File System Architecture

A file system is typically composed of three main layers or components:[6]

1. File Control Block (FCB)

The **File Control Block (FCB)** is a data structure maintained by the file system for each file.[7] It contains all the metadata about the file, similar to a file's "passport."[8] This metadata includes:

- **File Name**: The name of the file.[9]
- **File Size**: The size of the file in bytes.
- **Permissions**: Access rights (e.g., read, write, execute) for different users or groups.[10]
- **Creation, Access, and Modification Timestamps**: The dates and times when the file was created, last accessed, and last modified.[11]
- **Location Pointers**: Pointers to the disk blocks where the file's data is actually stored.

---

2. File Organization and Access Methods

The way a file system organizes and accesses data on disk defines its efficiency and performance.[12] There are several common methods:

- **Sequential Access**: Data is accessed in a sequential order from the beginning of the file.[13] This is common for tape storage and is simple to implement but inefficient for random access.[14]
- **Direct Access (Random Access)**: Data can be accessed directly from any location in the file.[15] This is crucial for databases and is possible on disk drives.
- **Indexed Access**: A file is a collection of pointers, called an index, to the actual data blocks.[16] This provides a balance between sequential and random access, as you can quickly find a data block using the index and then read it sequentially.

3. File Allocation Methods

The file system needs a way to allocate disk space to files.[17] The three most common methods are:

- **Contiguous Allocation**: Each file is stored in a single, contiguous block of disk space.[18] This is simple and provides fast access, as only one starting address is needed. However, it suffers from **external fragmentation**, where free space is scattered in small, unusable chunks.
- **Linked Allocation**: Each file is a linked list of disk blocks.[19] Each block contains a pointer to the next block.[20] This solves external fragmentation but has poor random-access performance because a file must be traversed from the beginning to find a specific block. It also has the overhead of storing pointers within each block.
- **Indexed Allocation**: Each file has a dedicated **index block** that contains pointers to all of its data blocks.[21] This is a very common method today. It offers fast direct access and eliminates external fragmentation. However, it requires a separate index block for each file, which can be an overhead for very small files.[22]

The file system's design, including its architecture, organization, and allocation methods, is critical for the overall performance, reliability, and security of an operating system.